Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

– FPT University, October 2026 –

Table of Contents

I. Record of Changes....3
II. Software Design Document....4
1. System Design....4
1.1 System Architecture....4
1.2 Package Diagram....4
2. Database Design....4
3. Detailed Design....5
3.1 <Feature/Function Name1>....5
3.2 <Feature/Function Name2>....6

I. Record of Changes

Date	A* M, D	In charge	Change Description
26/10/2026	M	Lê Nguyễn Quang Đức	Populate VISTA Review 2 content on original university template; code verification pending

*A - Added M - Modified D - Deleted

II. Software Design Document

1. System Design

Architecture separates browser, NestJS Education API (authorization/business logic), Supabase Auth/PostgreSQL, evidence storage, external AI/model service and local Python Robot Runtime. Drawings depict target boundaries; verify the current deployed state and API contracts before claiming integration.

1.1 System Architecture

Figure 01. VISTA_DIAGRAM.pdf — source page 1; conceptual / proposed unless code verified.

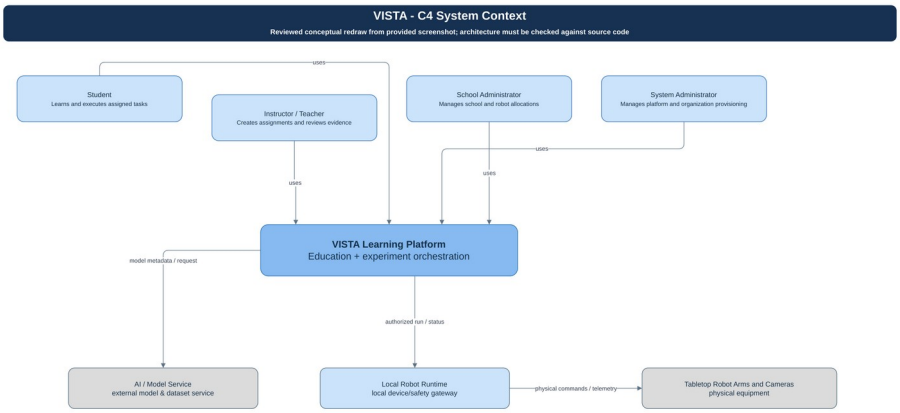


Figure 02. VISTA_DIAGRAM.pdf — source page 2; conceptual / proposed unless code verified.

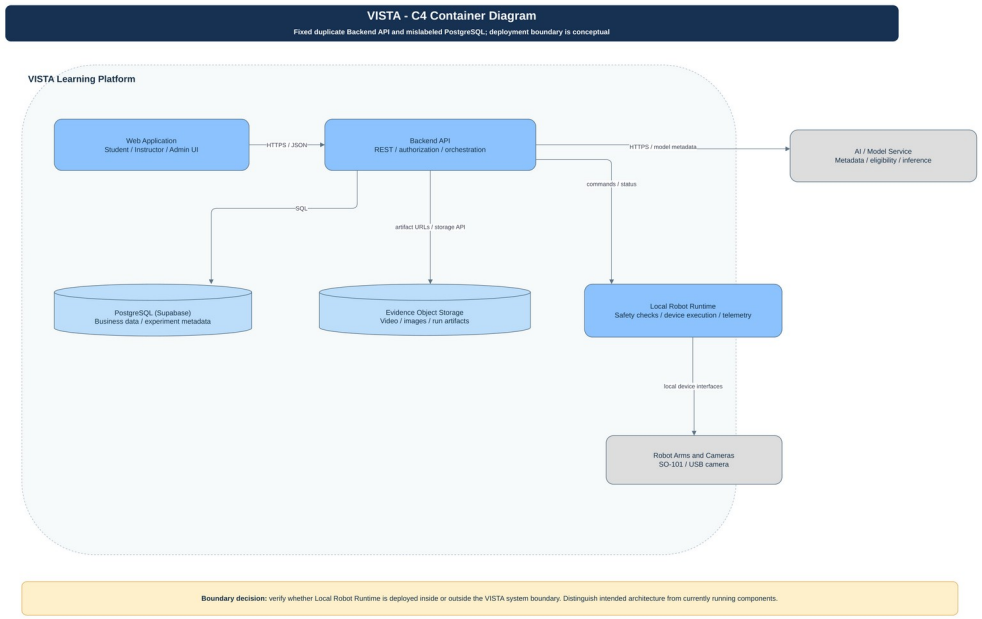
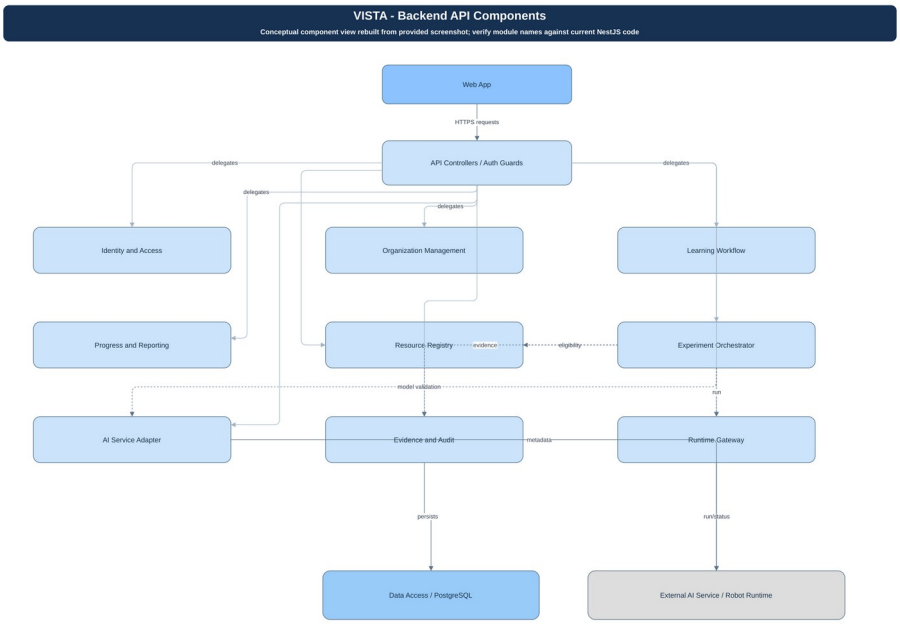


Figure 03. VISTA_DIAGRAM.pdf — source page 3; conceptual / proposed unless code verified.



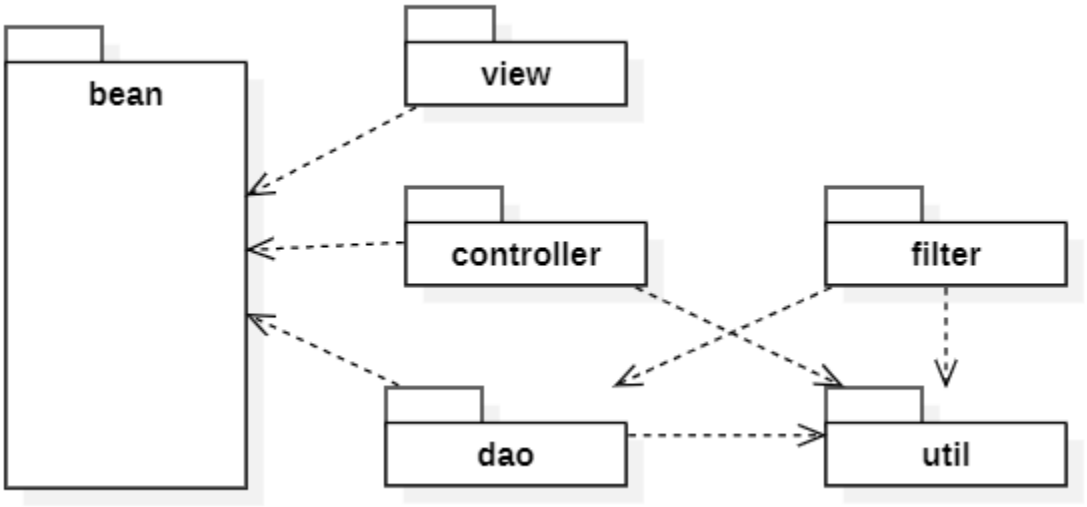
Source VISTA_DIAGRAM.pdf pages 1–3: context, container and conceptual backend components. Browser obtains JWT; NestJS guards validate membership and organization scope; Prisma persists records. Conceptual Experiment Orchestrator and Runtime Gateway must not be mistaken for verified NestJS module names.

C4 System Context: Student, Teacher, School/System Admin → VISTA Web → NestJS Education API → Supabase PostgreSQL/Auth; optional external AI/model service; Python Robot Runtime with fail-closed hardware safety. The submitted C4 drawings require verification against the repo before BM02.

1.2 Package Diagram

Mapped code roots: web/robot-edu (React/Vite); services/education-api (NestJS 11 and Prisma); services/ai-integration (AI interface); services/robot-runtime (Python); edge/raspberry-pi-agent (edge); services/backend requires independent inspection. Refer to embedded figures.

Logical packages: Identity/Access; Education Administration; Curriculum & Learning; Program Versioning; Experiment & Symbolic Evaluation; Assessment/Feedback; Robot Runtime Adapter; Persistence. These are design responsibilities, not necessarily separate deployed microservices.



Package Descriptions

No	Package	Description
1	Education API / access	NestJS controllers, guards, DTOs; Prisma data access
2	External integration	Python Robot Runtime and AI/model service; status unverified

2. Database Design

Figure 12. VISTA_DIAGRAM.pdf — source page 12; conceptual / proposed unless code verified.

12 Master Logical ERD

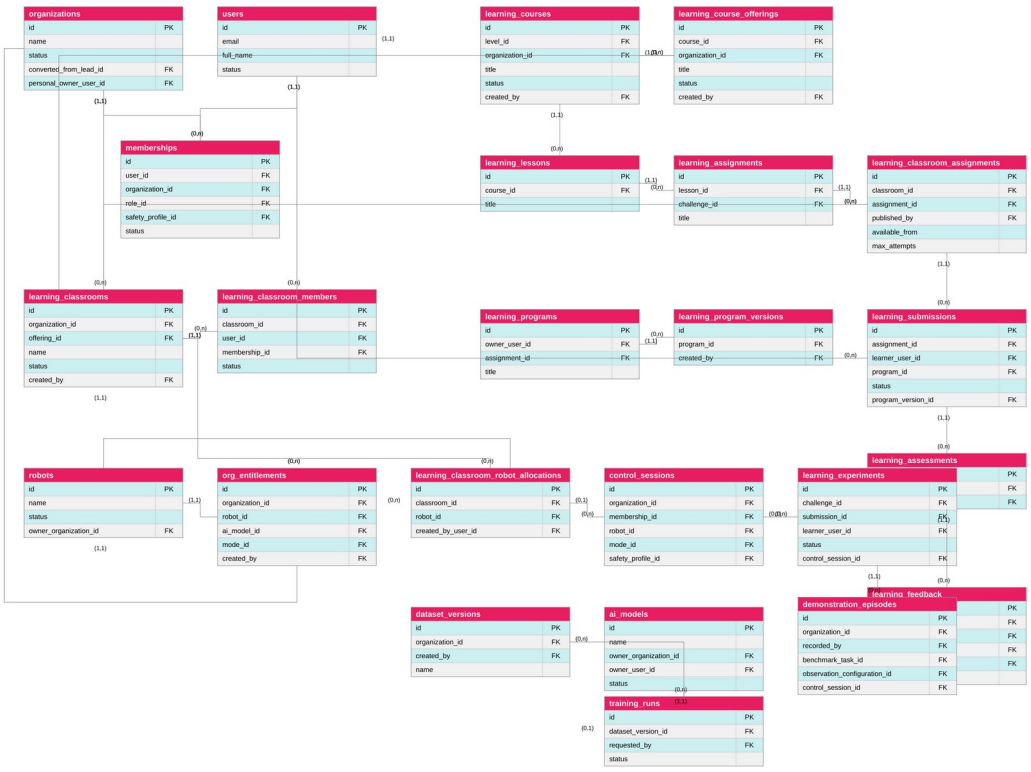


Figure 13. VISTA_DIAGRAM.pdf — source page 13; conceptual / proposed unless code verified.

13 ERD - Identity & School

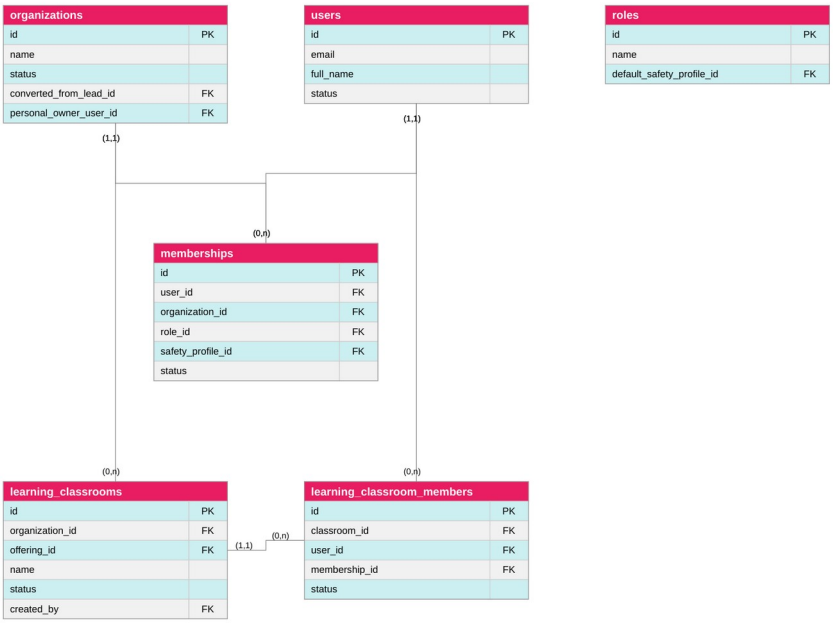


Figure 15. VISTA_DIAGRAM.pdf — source page 15; conceptual / proposed unless code verified.

14 ERD - Curriculum & Learning

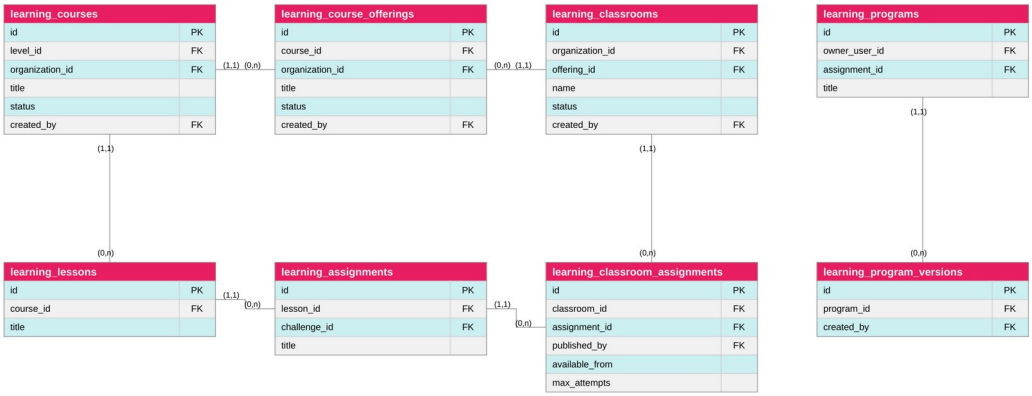


Figure 17. VISTA_DIAGRAM.pdf — source page 17; conceptual / proposed unless code verified.

15 ERD - Assessment

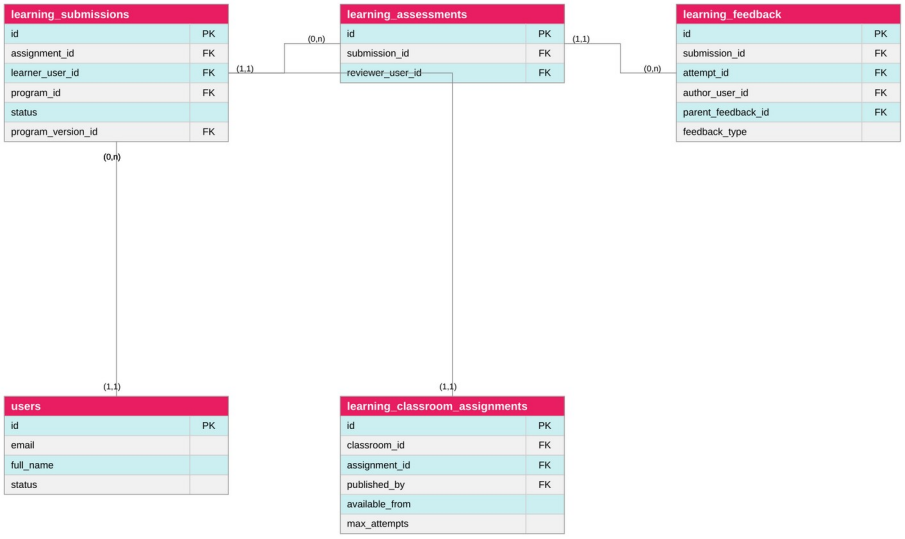


Figure 19. VISTA_DIAGRAM.pdf — source page 19; conceptual / proposed unless code verified.

16 ERD - Robot Resource Management

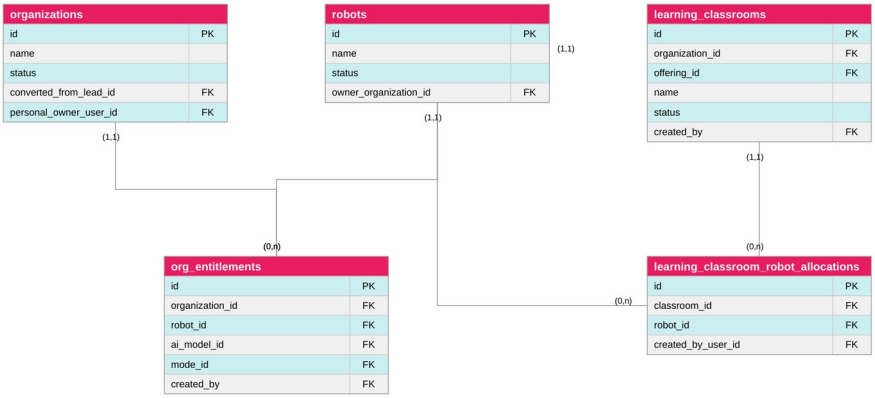


Figure 21. VISTA_DIAGRAM.pdf — source page 21; conceptual / proposed unless code verified.

17 ERD - Execution & Experiment

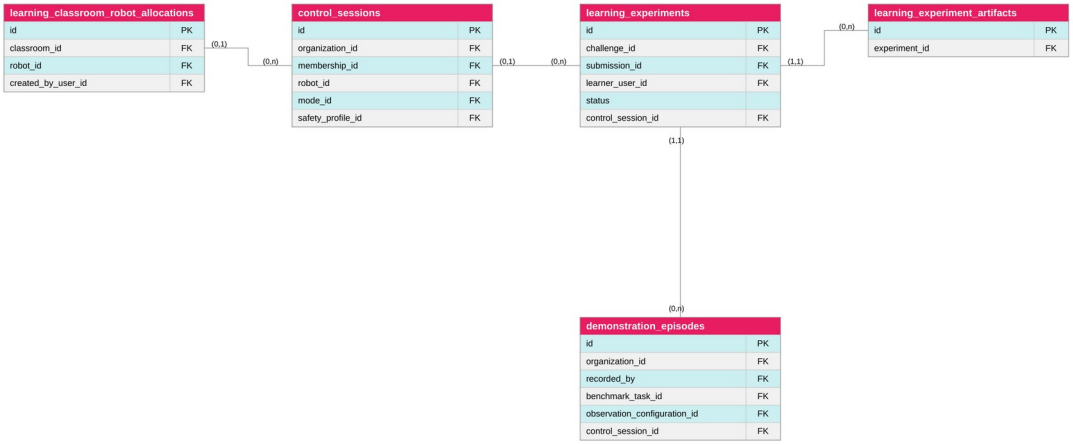
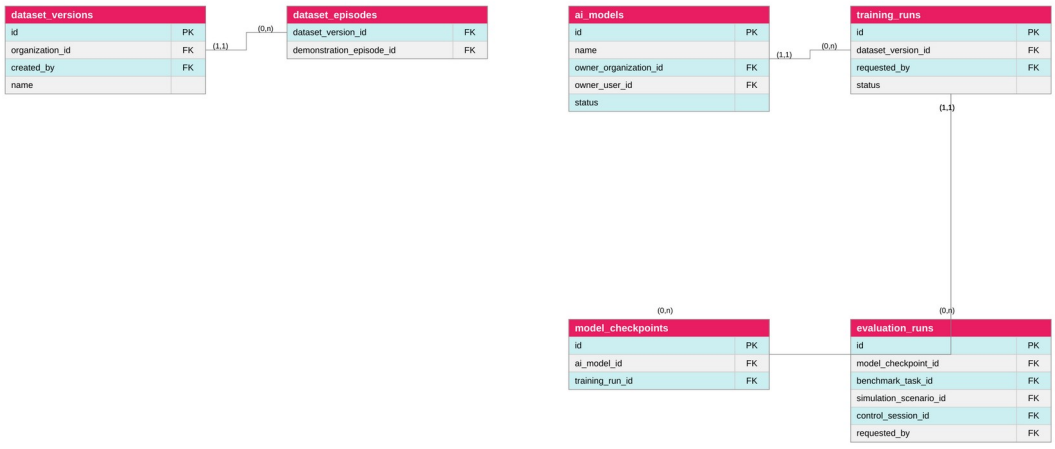


Figure 23. VISTA_DIAGRAM.pdf — source page 23; conceptual / proposed unless code verified.

18 ERD - AI & Datasets



Logical ERD approved visual in VISTA_DIAGRAM.pdf pages 12–23 (master and six domains). The image is a proposed logical design. Verify exact database constraints against Prisma schema and migration SQL prior to sign-off.

Database source of truth is applied PostgreSQL schema and migrations. Organization memberships, classroom membership, org_entitlements, classroom robot allocations, control_sessions, learning_experiments, submissions, program versions and assessments are distinct.

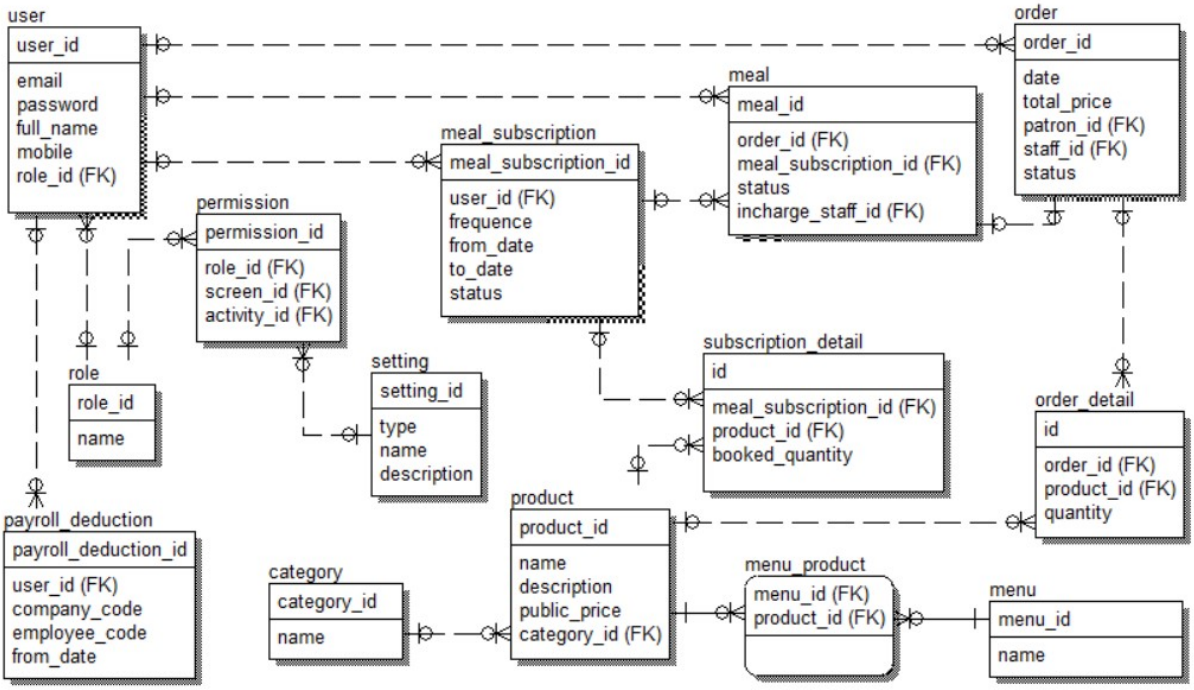


Table Descriptions

No	Table	Description
1	learning_classroom_robot_allocations	Robot authorization at classroom level
2	learning_experiments	Schematic experiment request/status; verify FK constraints

3. Detailed Design

Figure 04. VISTA_DIAGRAM.pdf — source page 4; conceptual / proposed unless code verified.

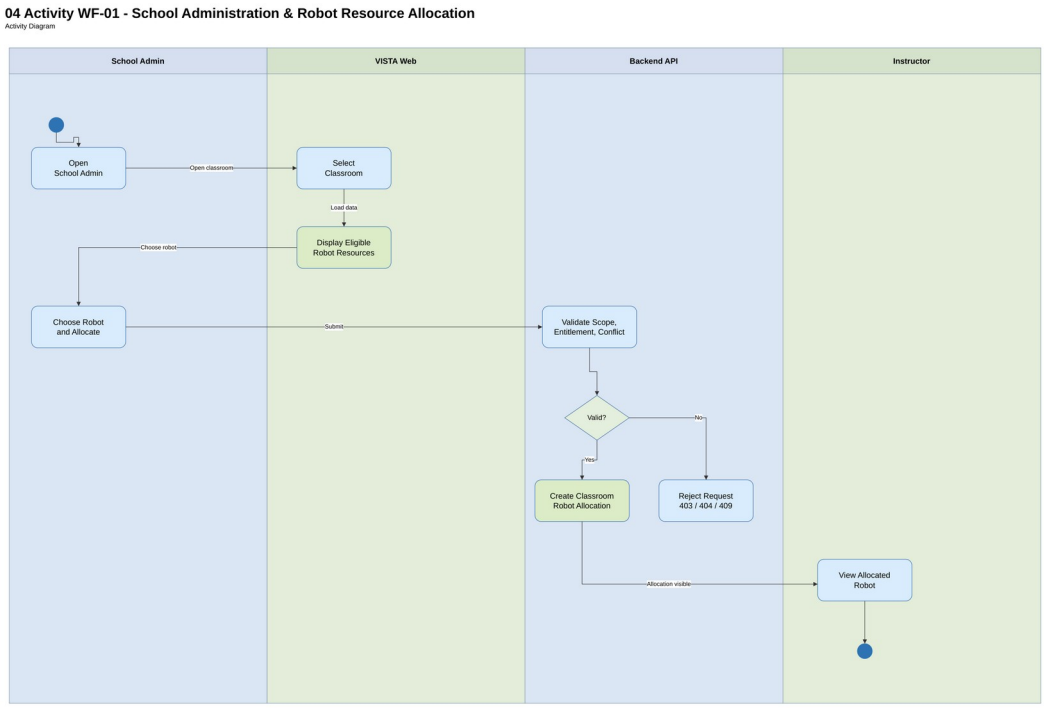


Figure 05. VISTA_DIAGRAM.pdf — source page 5; conceptual / proposed unless code verified.

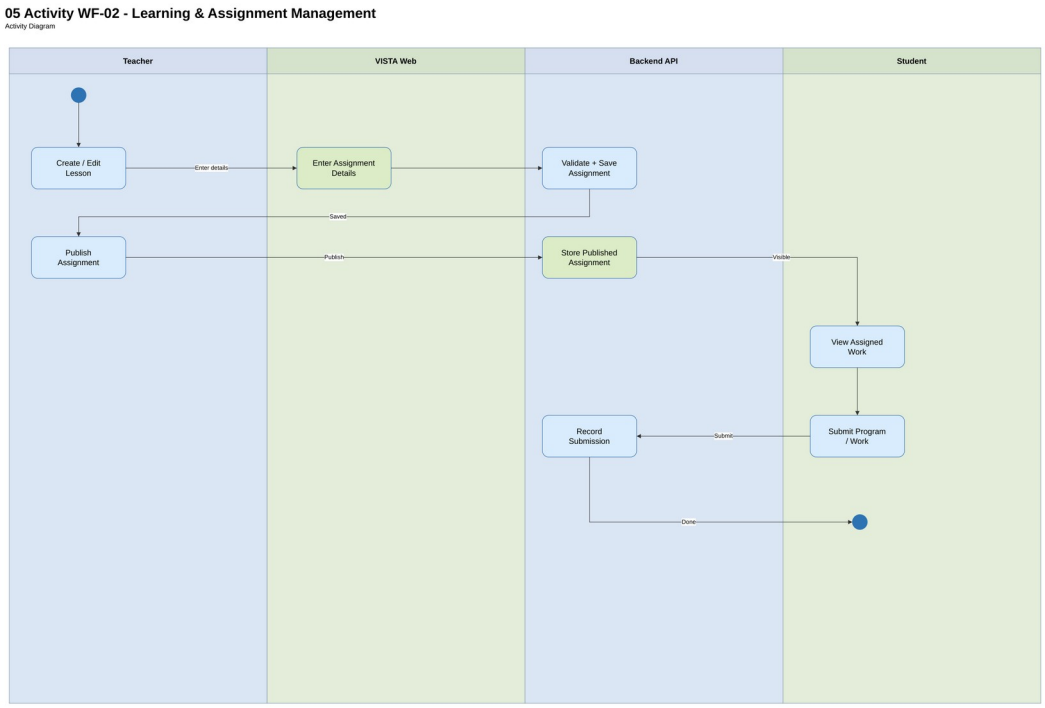


Figure 06. VISTA_DIAGRAM.pdf — source page 6; conceptual / proposed unless code verified.

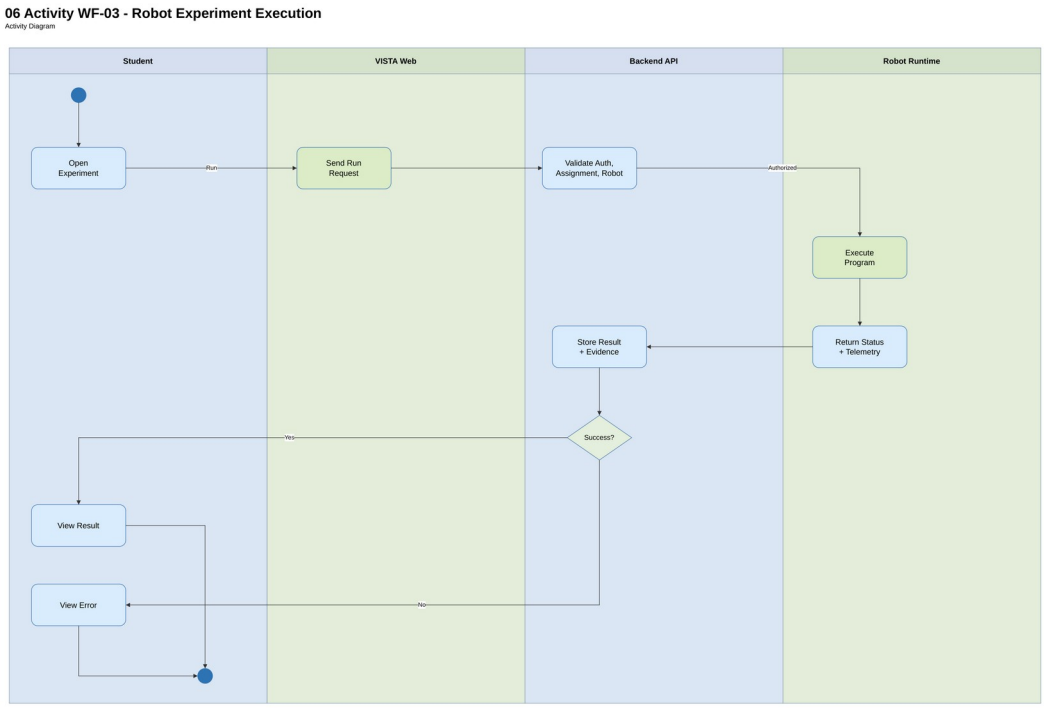


Figure 07. VISTA_DIAGRAM.pdf — source page 7; conceptual / proposed unless code verified.

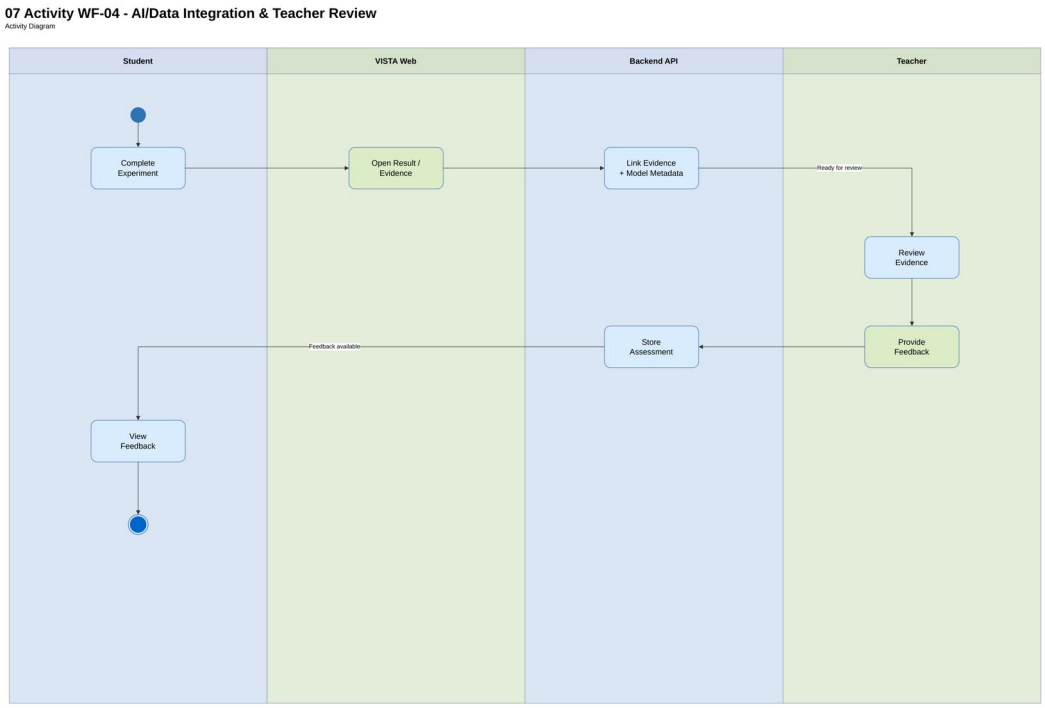


Figure 08. VISTA_DIAGRAM.pdf — source page 8; conceptual / proposed unless code verified.

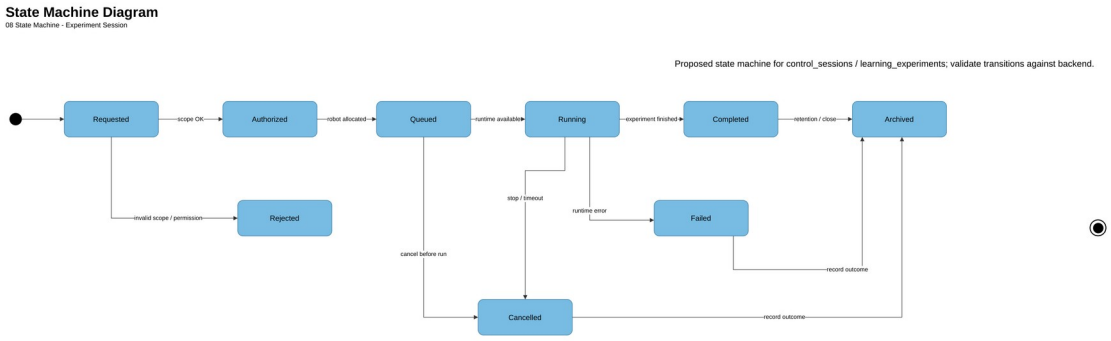


Figure 09. VISTA_DIAGRAM.pdf — source page 9; conceptual / proposed unless code verified.

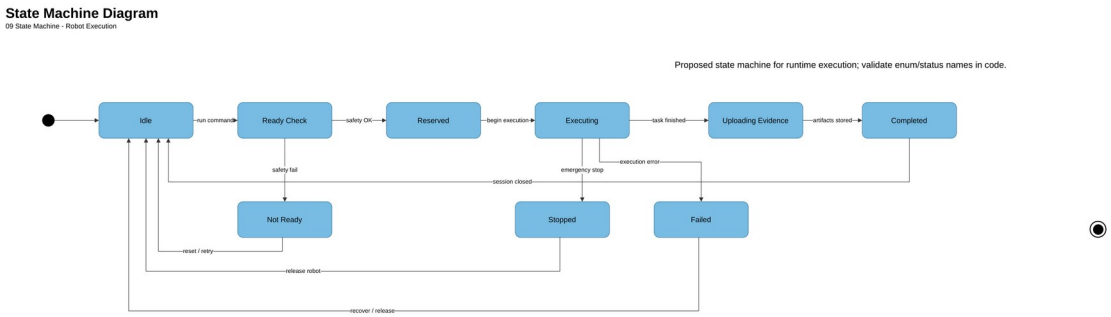


Figure 10. VISTA_DIAGRAM.pdf — source page 10; conceptual / proposed unless code verified.

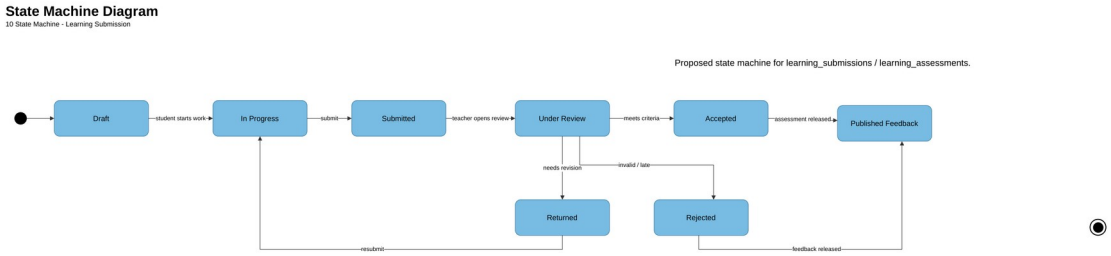
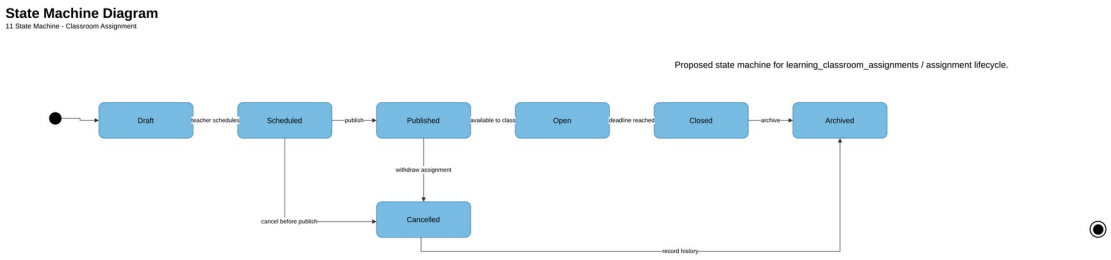


Figure 11. VISTA_DIAGRAM.pdf — source page 11; conceptual / proposed unless code verified.



Four workflows: WF-01 scoped allocation, WF-02 assignment/submission, WF-03 schematic experiment with proposed physical integration, WF-04 assessment/AI evidence. Workflow figures are embedded in this report.

3.1 WF-01 School Administration and Robot Allocation

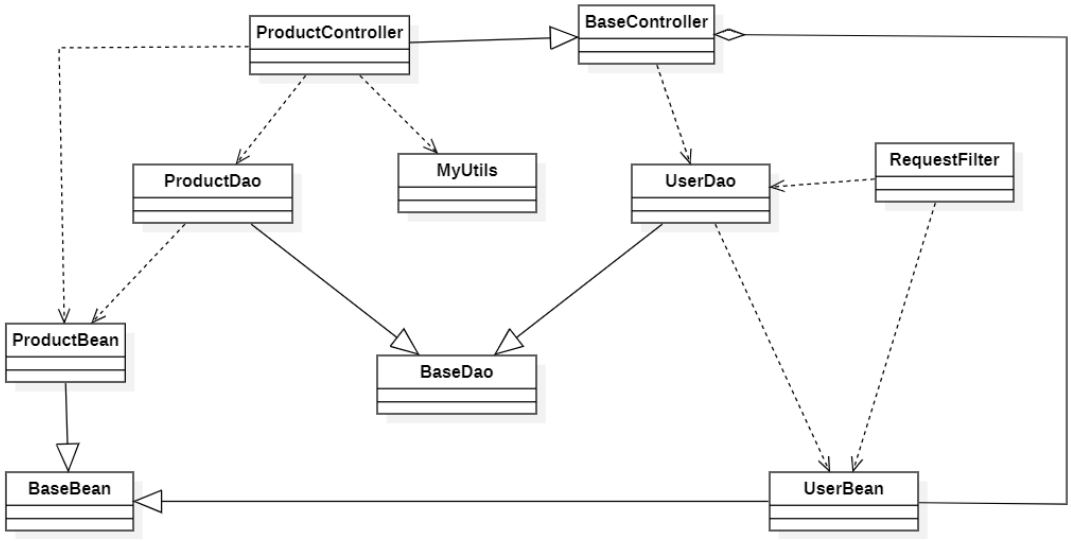
Validate Supabase JWT -> active membership -> classroom org scope -> robot entitlement -> atomic class allocation; no direct actuation or runtime reservation.

Controller: EducationAdminController; service: EducationAdminService. Guard maps Supabase JWT to VISTA user; scope checks active organization membership and entitlement; persistence updates one allocation per classroom atomically.

3.1.1 Class Diagram

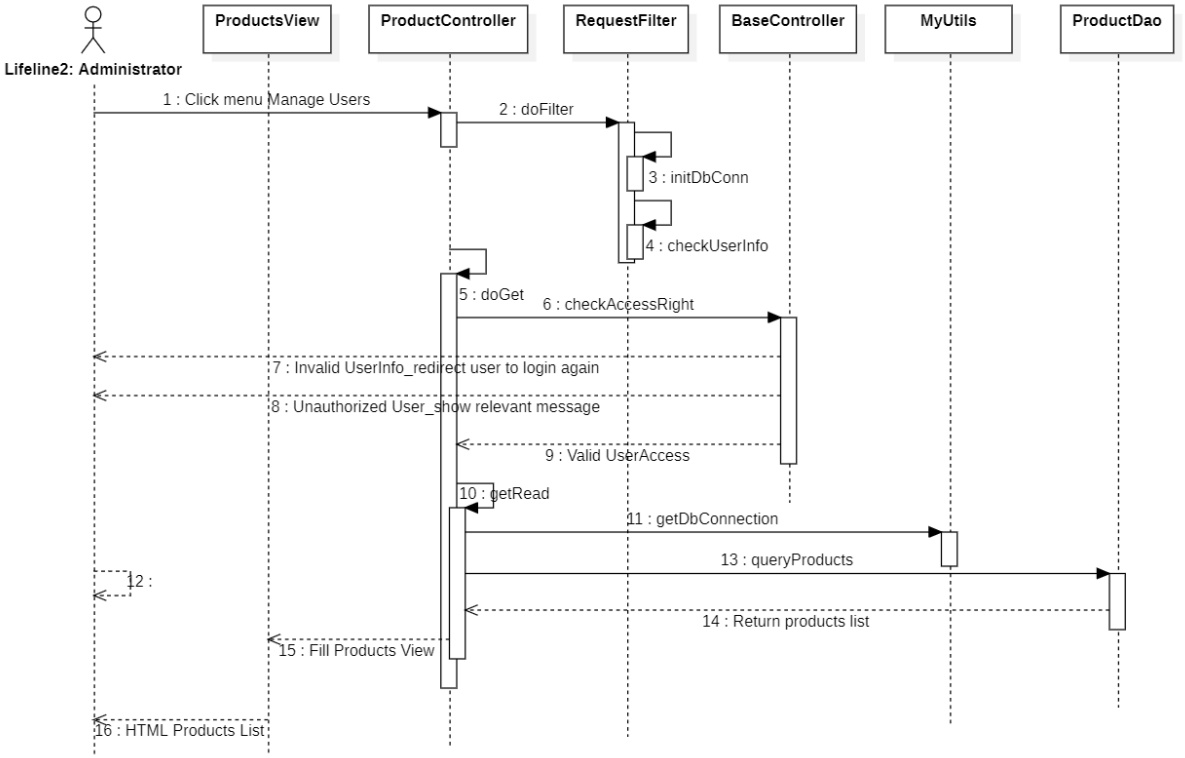
Relevant data model in Master ERD and Robot Resource ERD (figures below). Direct UML class implementation diagram must be generated/verified separately if required by school.

Class diagram: use existing services, DTOs, Prisma repositories; avoid modeling a new RobotAllocationService or a physical robot executor without implementation.



3.1.2 <Sequence Diagram Name1>

Sequence: School Admin → Web → EducationAdminController → scope checks → Prisma/DB → view allocation. Failure: deny inaccessible classroom/robot with no information leak.



3.1.3 WF-02–WF-04 Sequence Sources

Sequence diagrams are not present in the provided 18 diagrams. Proposed lifeline sequences must be verified against real controller/service code before marking as completed. The current architecture/activity/state/ERD diagrams are preserved without substitution.

Teacher creates classroom assignment → student starts attempt → program/version → submission. Student schematic experiment creates pending record; run-schematic/evaluate-symbolic are separate endpoints. Teacher assessment/feedback and student reflection are persisted separately.

3.2 State Machines / ERD / Deployment

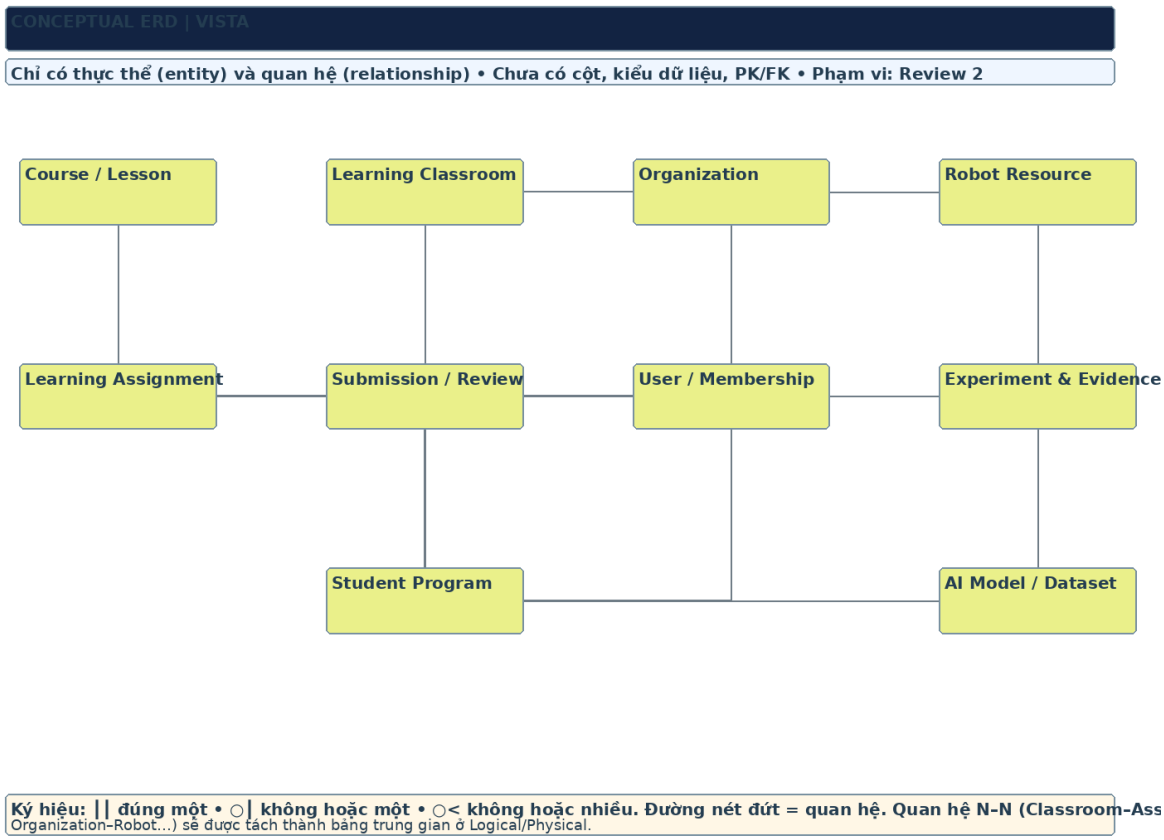
State machines in PDF pages 8–11 are PROPOSED (the original explicitly says validate statuses against backend). Do not use transition names as implemented enum claims. Development uses project scripts and Docker Compose where present. AWS is planned; deployment and rollback need actual staging evidence.

State machines must use code/database statuses, not visual filler. Master Logical ERD uses business FKs and junction entities. AWS architecture is a proposed staging design; not evidence of deployment.

Appendix — ERD diagrams from revised aldia.xml

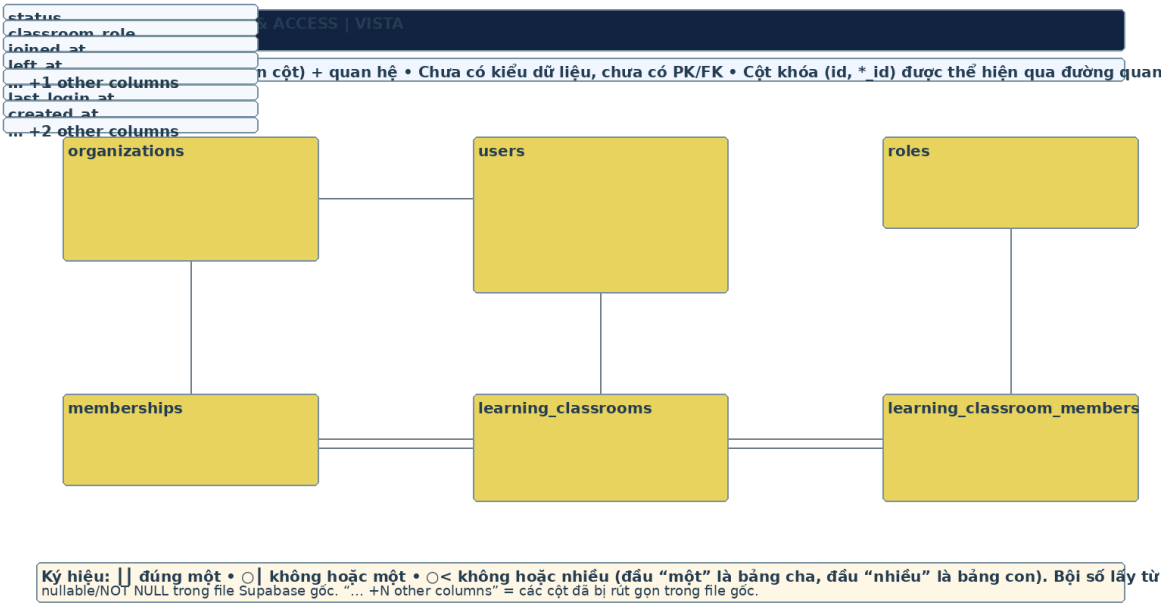
Source: user-supplied aldia.xml (22 editable draw.io pages). The following 11 images reproduce the Conceptual, Logical and Physical ERD views from that file. The editable XML is retained in 04_Diagrams. Keys, nullability, sizes and constraints require final checking against PostgreSQL migrations before final sign-off.

12. 12 Conceptual ERD



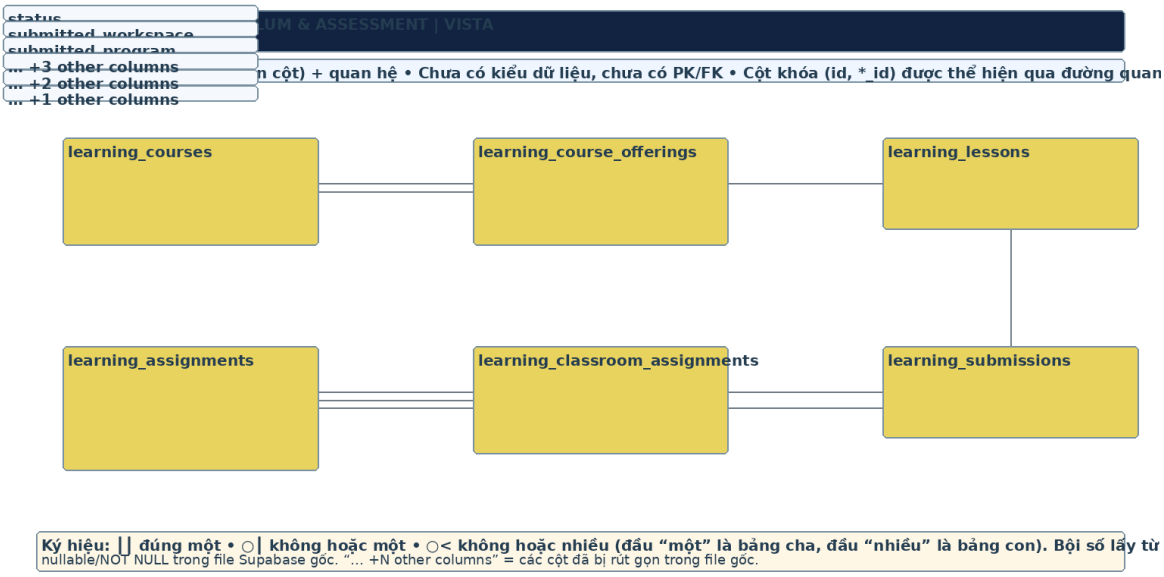
ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.

13. 13 Logical ERD — SCHOOL & ACCESS



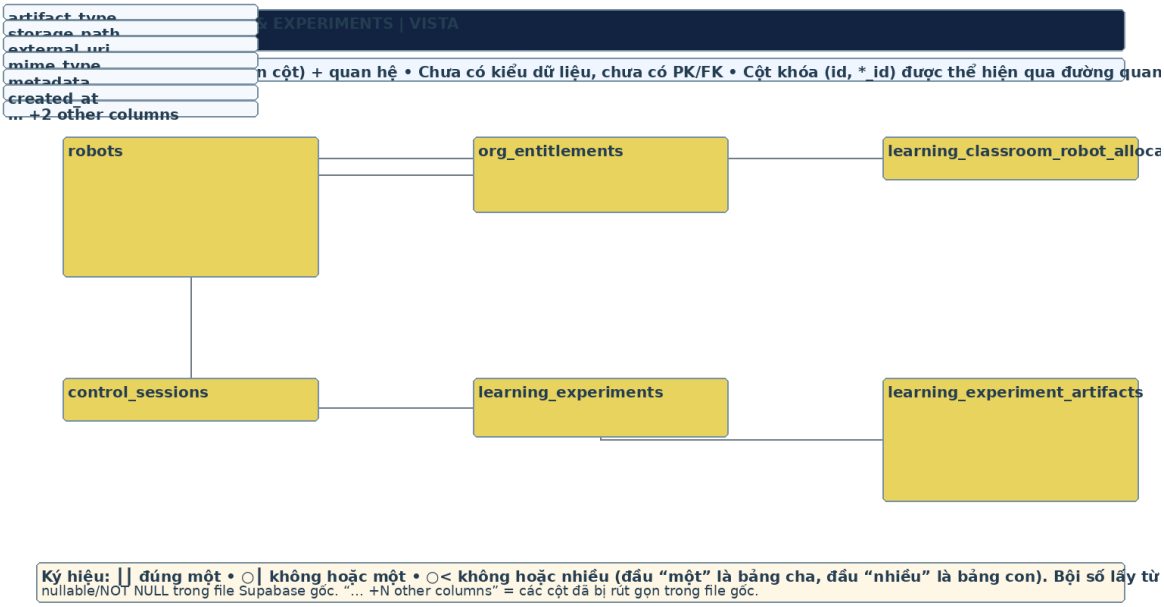
ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.

14. 14 Logical ERD — CURRICULUM & ASSESSMENT



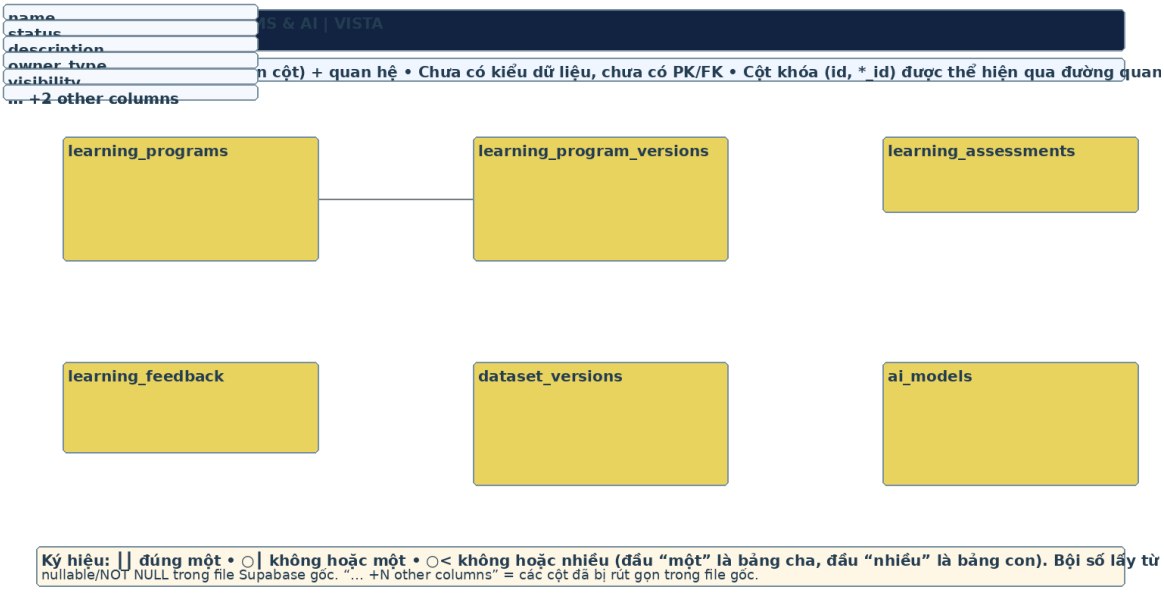
ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.

15. 15 Logical ERD — ROBOTS & EXPERIMENTS



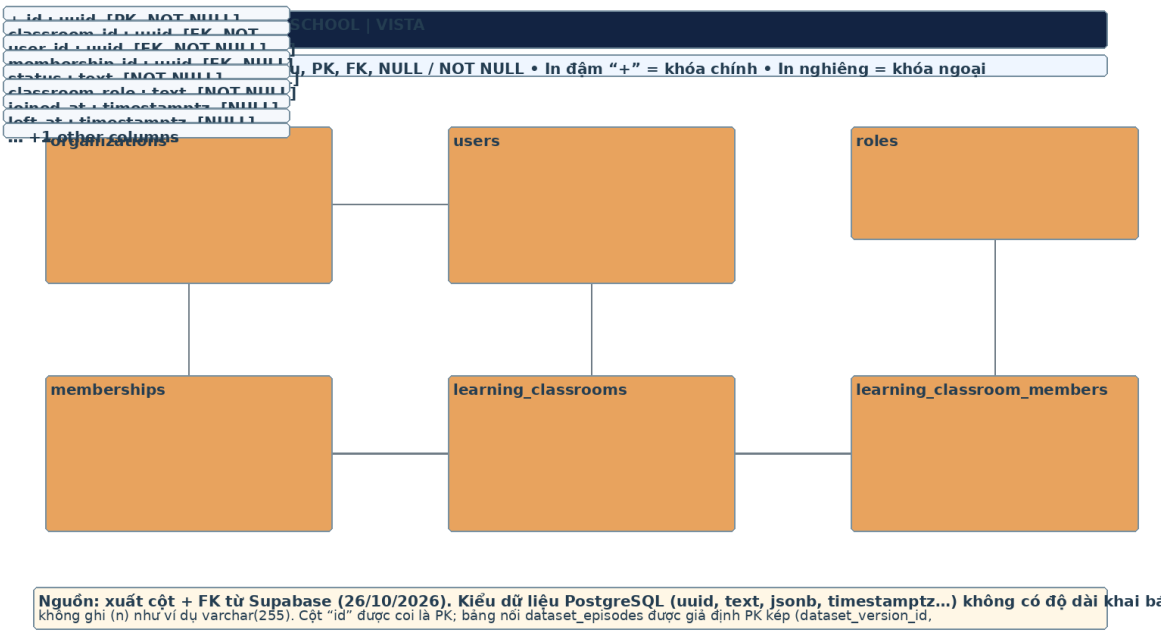
ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.

16. 16 Logical ERD — PROGRAMS & AI



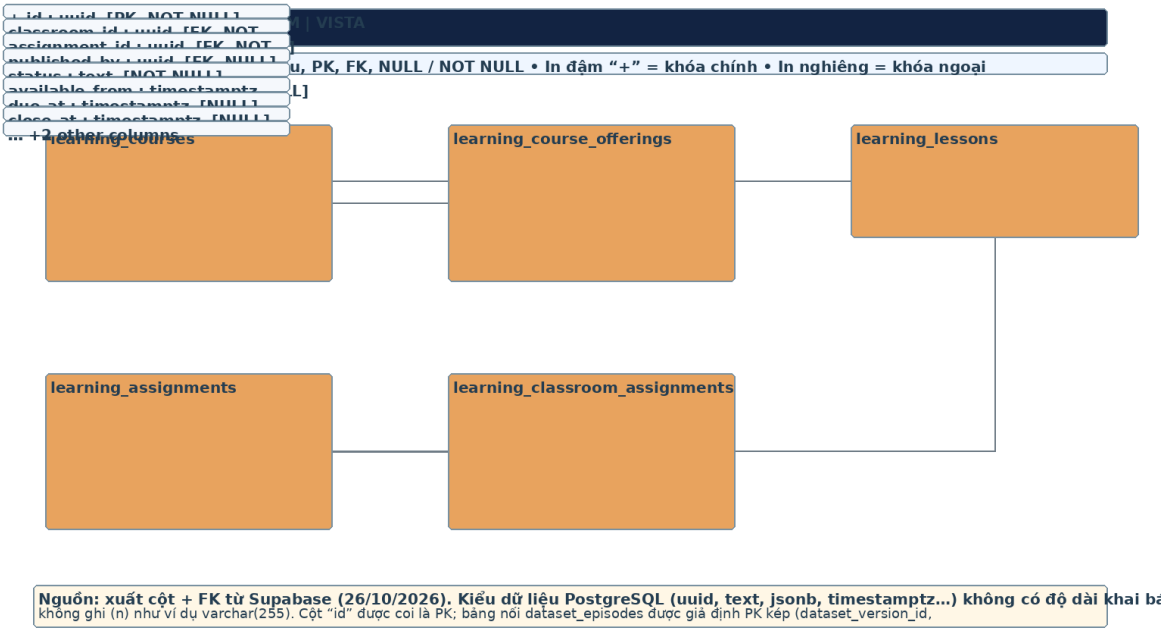
ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.

17. 17 Physical ERD — IDENTITY & SCHOOL



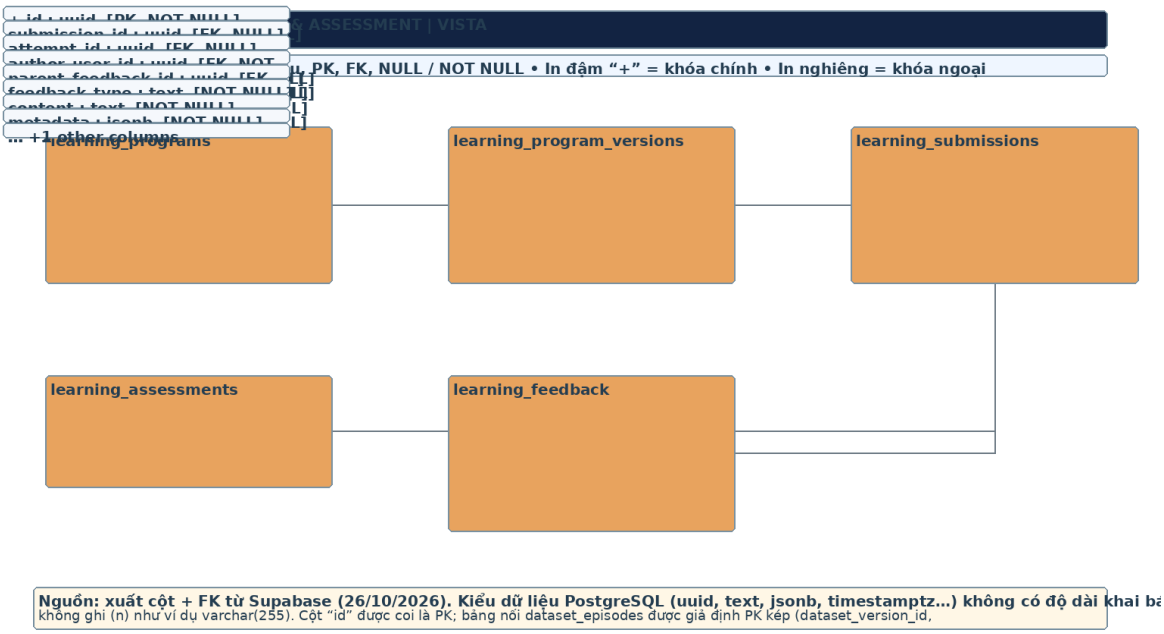
ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.

18. 18 Physical ERD — CURRICULUM



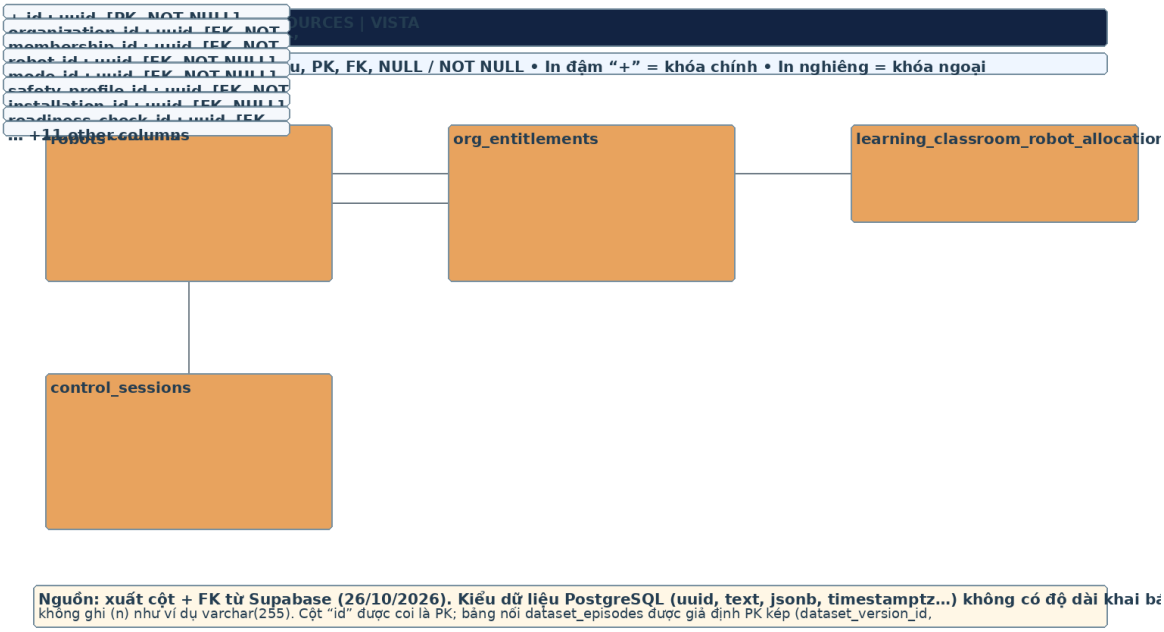
ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.

19. 19 Physical ERD — PROGRAMS & ASSESSMENT



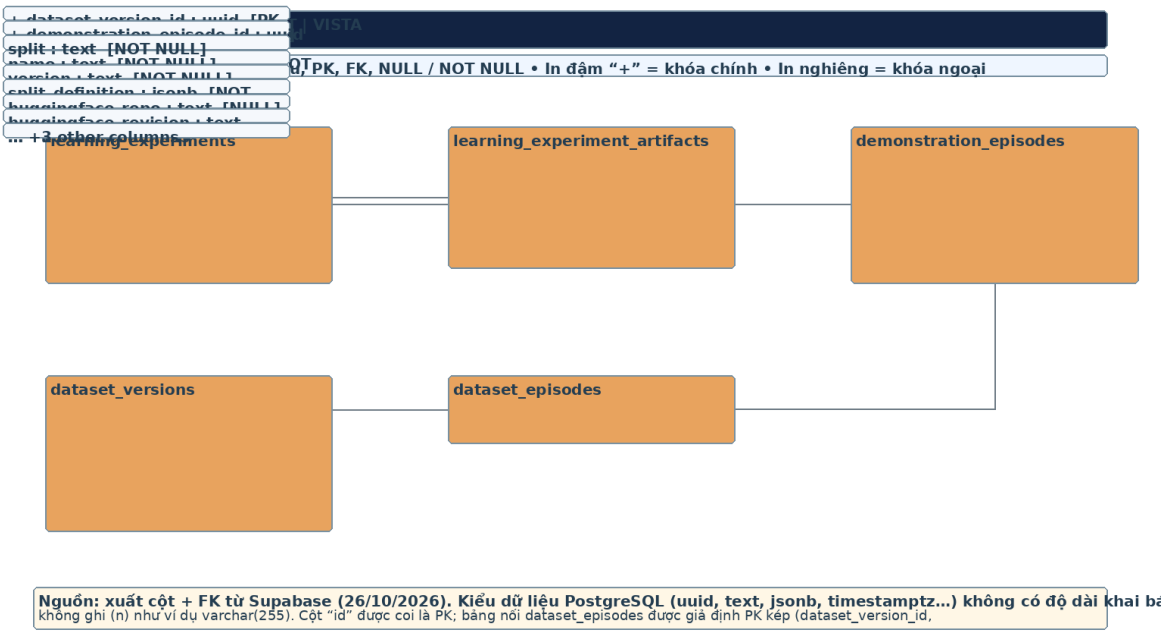
ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.

20. 20 Physical ERD — ROBOT RESOURCES



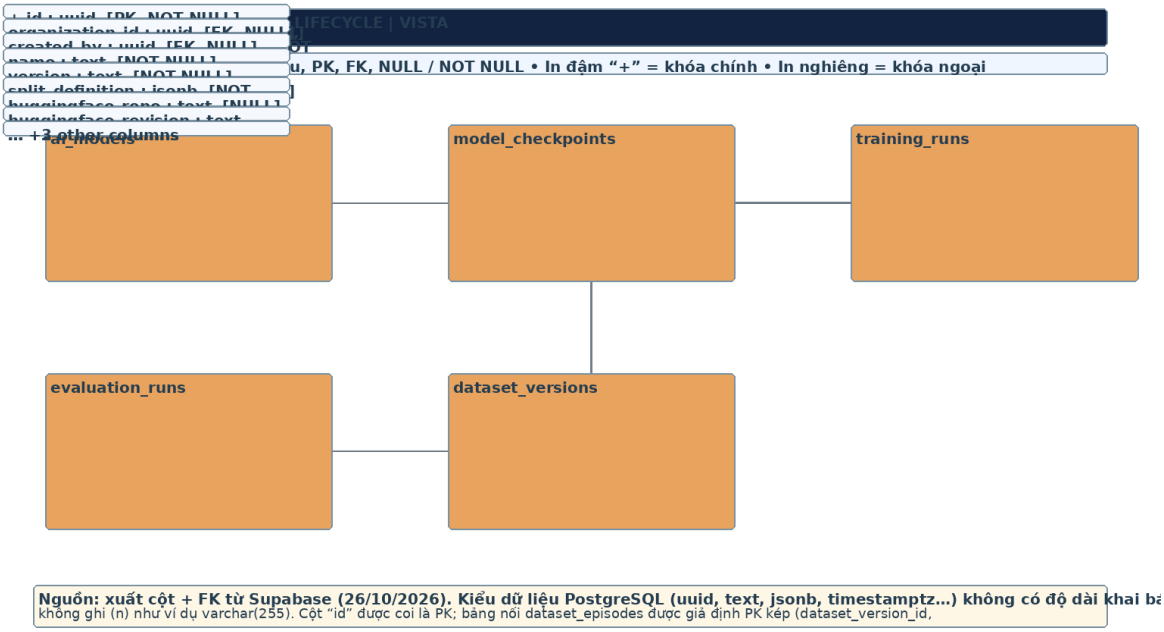
ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.

21. 21 Physical ERD — EXPERIMENT



ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.

22. 22 Physical ERD — AI & MODEL LIFECYCLE



ERD reference — editable source: aldia.xml; physical constraints must be verified against migrations.